

Lecture 05: Encapsulation and static

Introduction to Programming with Java

Today's learning goals

By the end of this lecture you should be able to:

1. Explain why we hide fields with `private`
2. Use `public` methods as a controlled interface (getters/setters)
3. Contrast **instance** members with **static** members
4. Decide when `static` is appropriate at a beginner level

Agenda

1. Why encapsulation?
2. `public` vs `private`
3. Getters and setters
4. Protecting invariants (simple validation)
5. `static` fields and methods (light)
6. `main` revisited

The problem with public fields

If every field is freely writable from outside:

- Callers can set nonsense values (`balance = -1000000`)
- Hard to change representation later (break all client code)
- Object rules are scattered across the program

Encapsulation in one sentence

Idea

Keep the object's **data private**; allow access only through well-chosen **public methods**.

Think of a drinks machine: you use buttons (methods), not the internal wiring (fields).

Visibility modifiers (intro)

- `public` — accessible from other classes
- `private` — accessible only inside this class

(There are more levels later: `package-private`, `protected`. Not needed today.)

Before / after

```
1 // Fragile: anyone can break the rules
2 public double balance;
3
4 // Better: hide the field
5 private double balance;
6
7 public double getBalance() {
8     return balance;
9 }
```

Quiz 1 — Encapsulation

Quiz

What does making a field `private` mainly achieve?

1. Faster programs always
2. The field can only be used inside its own class (controlled access)
3. The field is deleted at runtime
4. The class can no longer have methods

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Private fields are hidden from other classes; use methods to interact safely.

- **Getter:** returns a field value (read)
- **Setter:** updates a field (write), often with checks
- Not every field needs a setter (immutable style is fine)

BankAccount sketch

```
1 public class BankAccount {  
2     private double balance;  
  
3  
4     public BankAccount(double start) {  
5         if (start < 0) {  
6             start = 0;  
7         }  
8         balance = start;  
9     }  
10  
11     public double getBalance() {  
12         return balance;  
13     }
```

Methods enforce rules

```
1 public void deposit(double amount) {  
2     if (amount > 0) {  
3         balance += amount;  
4     }  
5 }  
6  
7 public boolean withdraw(double amount) {  
8     if (amount > 0 && amount <= balance) {  
9         balance -= amount;  
10        return true;  
11    }  
12    return false;  
13 }  
14 }
```

Why not a public setter for balance?

- `setBalance(1e9)` would skip deposit rules
- Prefer verbs that match real actions: `deposit`, `withdraw`
- Getters are often fine for reading current state

Try this

Add a `Student` with private `name` and `id`, a constructor, getters, and `setName` that rejects blank names.

Invariants

An **invariant** is a rule that should always stay true, e.g.:

- Balance never negative (for this simple account)
- Id always positive

Private fields + careful methods help you keep invariants.

Quiz 2 — What is allowed?

```
1 // inside another class:  
2 BankAccount a = new BankAccount(50);  
3 a.balance = -10;    // ?
```

Quiz

If balance is private, this line ...

1. Works and sets balance to -10
2. Does not compile (not accessible)
3. Runs but prints a warning only
4. Deletes the account

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Does not compile — private fields are not accessible from other classes.

Two kinds of members

Instance

Belongs to each object

Fields: per-object state

Methods: need an instance

static

Belongs to the class

Shared among all objects

Call via `ClassName.method`

static method example

```
1 public class MathUtil {  
2     public static int clamp(int value, int lo, int hi) {  
3         if (value < lo) return lo;  
4         if (value > hi) return hi;  
5         return value;  
6     }  
7 }  
8  
9 // usage -- no object required:  
10 int x = MathUtil.clamp(150, 0, 100); // 100
```

static field: shared counter

```
1 public class User {  
2     private static int nextId = 1; // shared  
3     private int id;  
4     private String name;  
5  
6     public User(String name) {  
7         this.name = name;  
8         this.id = nextId;  
9         nextId++;  
10    }  
11  
12    public int getId() { return id; }  
13 }
```

When to use static (beginner guidance)

- Helpers that need **no object state** (`Math.max`, utilities)
- Shared constants: `public static final double PI = ...` (concept)
- `main` must be static — JVM starts with no instance

Prefer instance methods when behaviour depends on an object's fields.

Common confusion

```
1 public class Counter {  
2     private int value;  
3  
4     public static void brokenHit() {  
5         // value++; // ERROR: cannot access instance field  
6     }  
7  
8     public void hit() {  
9         value++; // OK  
10    }  
11 }
```

Common mistake

Putting all logic in static methods and then wondering why fields are inaccessible.

- `public static void main(String[] args)`
- `static`: called without creating your class first
- Typical pattern: `main` creates objects, then calls instance methods

Quiz 3 — static vs instance

Quiz

You need a method that uses a particular `BankAccount`'s balance. It should usually be:

1. `static`, called as `BankAccount.deposit(50)`
2. An instance method, called as `account.deposit(50)`
3. Outside any class
4. Named `main`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Instance method — it depends on that account's state.

Quiz

True or false?

A `static` method can freely read an object's private instance fields without having a reference to an object.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

False. Static methods have no `this`; they need an object reference to touch instance fields (and privacy rules still apply).

Key takeaways

1. Encapsulation: hide data (`private`), expose behaviour (`public` methods)
2. Getters/setters (or action methods) protect rules and invariants
3. Instance members belong to objects; `static` belongs to the class
4. Prefer instance methods for object state; `static` for utilities / `main`

Coming next

Lecture 06

ArrayList for growable lists of objects

A short OOP practice wrap combining classes + collections

Private fields or `static`—which still feels fuzzy?